PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II

DISUSUN OLEH :

NAMA : Muhammad Daffa Al Barru

NPM : 25782017

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

MENTRI PENDIDIKAN DAN KEBUDAYAAN PROVINSI LAMPUNG

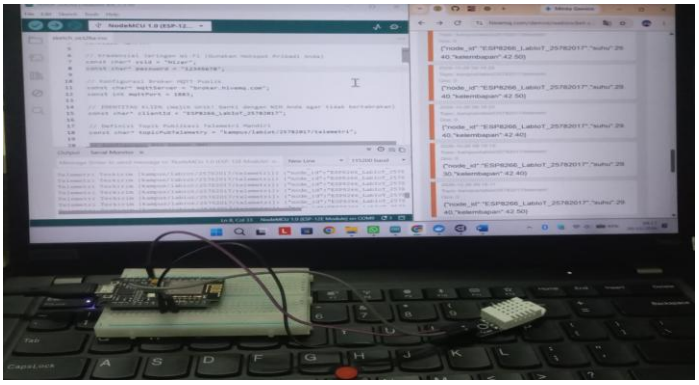
POLITEKNIK NEGERI LAMPUNG

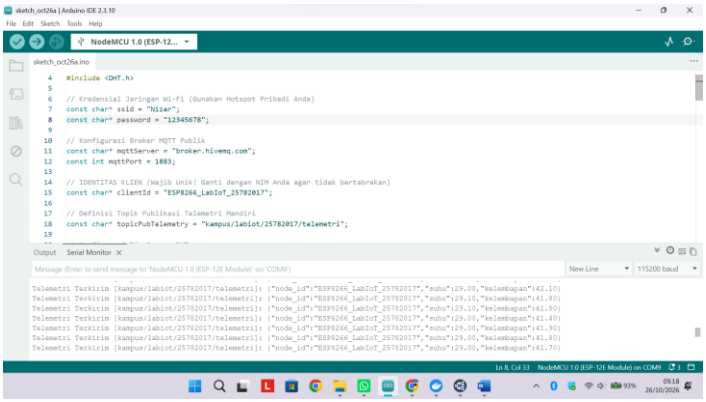
TAHUN AJARAN 2025 / 2026

6.5 Praktikum

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

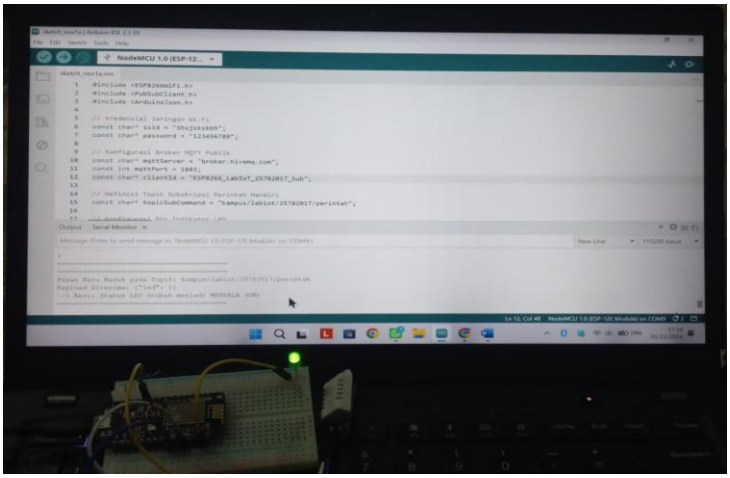
Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Publisher*. Perangkat membaca data suhu dan kelembapan dari sensor DHT11/DHT22, mengemas parameter tersebut ke dalam dokumen JSON menggunakan ArduinoJson, lalu memublikasikannya setiap 5 detik ke broker publik broker.hivemq.com.

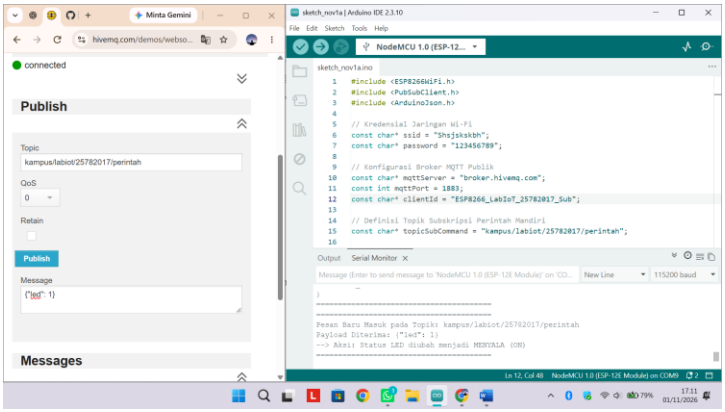




6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

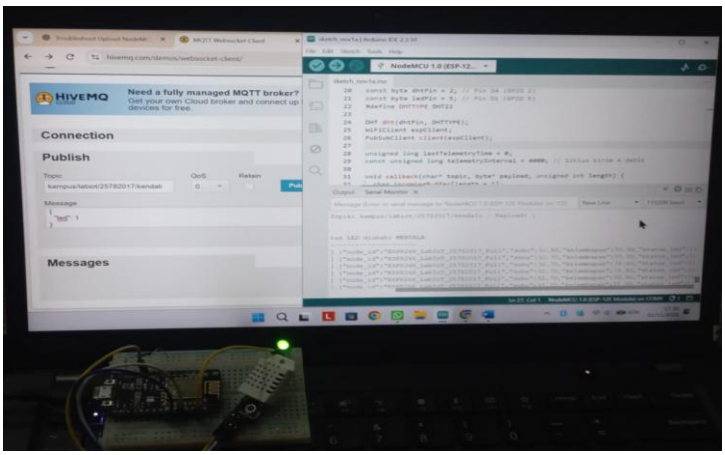
Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

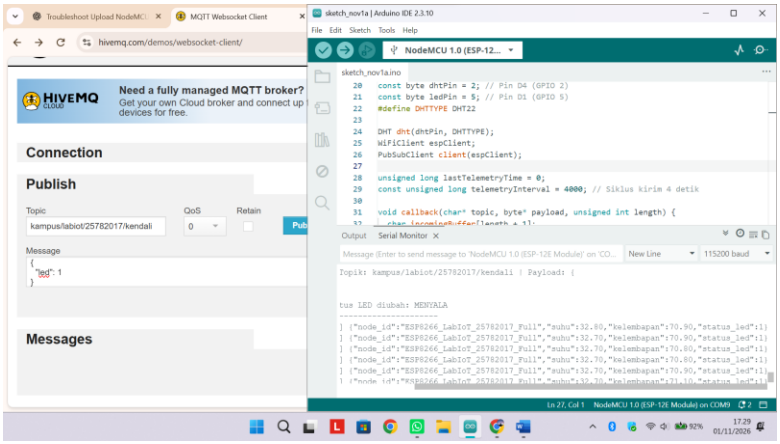




6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.





6.6 Latihan

- **Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):**
  Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

  i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler. MQTT memiliki *header* minimal (hanya 2 byte), sedangkan HTTP membawa *header* teks panjang (200–800 byte) per permintaan, sehingga MQTT jauh lebih hemat kuota.

  ii. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah. MQTT mengelola koneksi dan *ping* secara asinkron tanpa membebani mikrokontroler. HTTP bersifat *stateless* yang mengharuskan pembukaan/penutupan koneksi berulang, memicu konsumsi CPU dan daya tinggi.

  iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*). MQTT mendukung pengiriman ke banyak *subscriber* sekaligus secara otomatis melalui *broker*. HTTP bersifat *point-to-point*, sehingga harus mengirim data satu per satu ke tiap klien (menyebabkan *bottleneck*).

- **Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):**
  Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

  i. Memantau parameter suhu dari seluruh mesin di semua lini produksi. Menggunakan tanda + (*single-level wildcard*) untuk menggantikan posisi lini (line1/line2/line3) dan mesin (m1/m2/m3/m4), serta memfokuskan akhir topik pada sensor suhu.

ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2. Menetapkan jalur spesifik pada line2/m1 dan menggunakan tanda # (*multi-level wildcard*) di akhir untuk menangkap seluruh sub-topik parameter sensor di bawahnya.

iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut. Menggunakan tanda # di akhir *root* pabrik/ untuk menyadap seluruh pesan dan hierarki turunan apa pun yang mengalir di dalam sistem pabrik.

- **Latihan 3 (Implementasi Serialisasi Data Dinamis):**
  Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

  i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).

  ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
  Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

  Tambahkan baris berikut pada fungsi pembentukan JSON:

  ```
  doc["uptime_detik"] = millis() / 1000;

  doc["ip_lokal"] = WiFi.localIP().toString();
  ```

- **Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):**
  Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

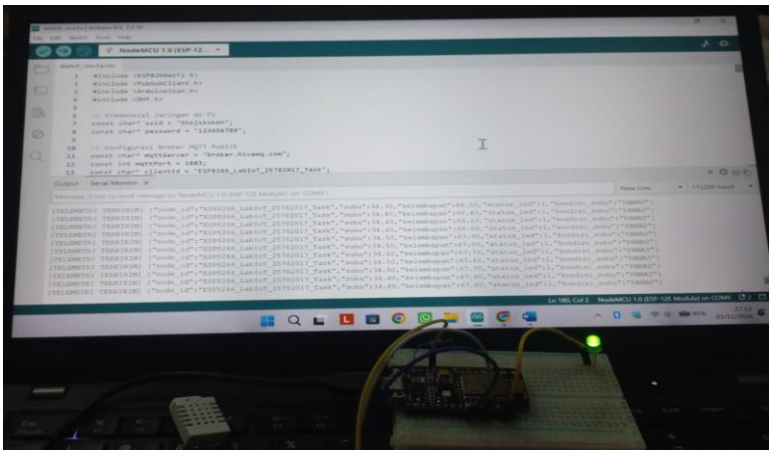
  i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()? Mikrokontroler akan terblokir total selama 6 detik. Fungsi client.loop() gagal berjalan, membuat perangkat tidak responsif terhadap perintah masuk dan berisiko terputus dari *broker* akibat gagal mengirim *keep-alive ping*.

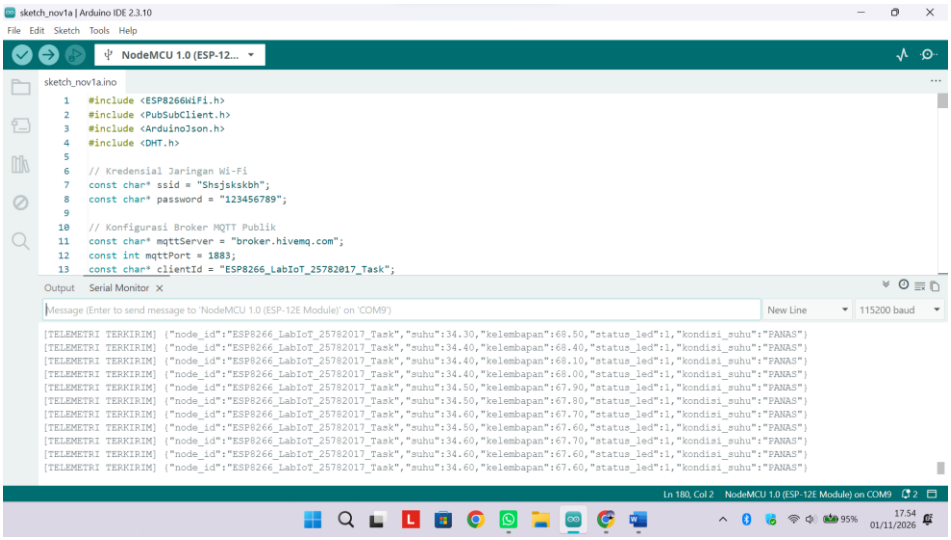
  ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat! Fungsi mengecek client.connected(). Jika bernilai *false*, sistem mencoba jabat ulang (client.connect) dan langsung mendaftarkan ulang subskripsi topik (client.subscribe) agar perangkat kembali aktif tanpa *reboot* manual.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

Kembangkan implementasi Praktikum 3 menjadi sistem pengawasan keselamatan dan proteksi otomasi industri tertutup (*Closed-Loop Industrial Safety & Automated Actuation*) yang tangguh dan cerdas secara mandiri:

1. **Analisis Telemetri dan Klasifikasi Kondisi Lingkungan:** Mikrokontroler secara periodik membaca parameter suhu dan kelembapan, lalu membungkusnya ke dalam format JSON. Berdasarkan *Rule Engine* yang diterapkan, sistem secara cerdas menyematkan parameter "kondisi_suhu" bernilai "NORMAL" ketika suhu terukur $\le 31.0^\circ\text{C}$ dan otomatis berubah menjadi "PANAS" saat suhu melampaui ambang batas 31.0°C
2. **Mekanisme *Closed-Loop* dan *Dedicated Alert Topic*:** Ketika sensor mendeteksi kondisi *overheat* ($\text{suhu} > 31.0^\circ\text{C}$), mikrokontroler tidak hanya bergantung pada instruksi luar, melainkan langsung mengeksekusi proteksi lokal (*edge safety*). Sistem secara asinkron memublikasikan paket peringatan darurat ke topik kampus/labiot/nim_anda/alert dengan status "BAHAYA" serta mengaktifkan LED indikator secara fisik (*hardware override*) seketika. Implementasi *state latch* memastikan pengiriman pesan *alert* tetap teratur dan tidak membanjiri *broker*
3. **Kendali Manual (*Manual Override*) dan Asinkronitas *Callback*:** Saat lingkungan berada dalam status normal, operator luar tetap dapat mengambil alih kendali aktuator pencahayaan/indikator secara jarak jauh (*manual override*) melalui *dashboard* MQTT Explorer dengan mengirimkan payload JSON {"led": 1} atau {"led": 0} ke topik kampus/labiot/nim_anda/kendali . Proses penerimaan perintah ini ditangani secara non-blokir oleh fungsi *callback* tanpa mengganggu siklus pengiriman data telemetri utama.
4. **Keandalan Jaringan dan Protokol LWT (*Last Will and Testament*):** Untuk menjamin visibilitas status perangkat secara *real-time*, sistem dikonfigurasikan dengan protokol LWT pada topik kampus/labiot/nim_anda/status. Sesaat setelah terhubung, perangkat memublikasikan pesan status "ONLINE". Berdasarkan pengujian pemutusan daya mendadak (kabel USB dicabut), *MQTT Broker* secara otomatis menerbitkan pesan wasiat berformat JSON bernilai "OFFLINE", sehingga operator pusat dapat langsung mendeteksi kegagalan perangkat tanpa harus menunggu waktu kedaluwarsa sesi TCP habis




sketch_nov1a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_nov1a.ino
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>
// Kredensial Jaringan Wi-Fi
const char* ssid = "Shsjskskbh";
const char* password = "123456789";
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782017_Task";
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM9')
New Line
115200 baud
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.30,"kelembapan":68.50,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.40,"kelembapan":68.40,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.40,"kelembapan":68.10,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.40,"kelembapan":68.00,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.50,"kelembapan":67.90,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.50,"kelembapan":67.80,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.60,"kelembapan":67.70,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.50,"kelembapan":67.60,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.60,"kelembapan":67.70,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.60,"kelembapan":67.60,"status_led":1,"kondisi_suhu":"PANAS"}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782017_Task","suhu":34.60,"kelembapan":67.60,"status_led":1,"kondisi_suhu":"PANAS"}
Ln 180, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM9
95%
17.54
01/11/2026